



# Introduction to NitroStack

Welcome to NitroStack. This guide details the architecture and capabilities of the framework designed for building production-grade Model Context Protocol (MCP) servers.

## Overview

NitroStack is a high-performance TypeScript framework engineered for scalability and maintainability. It provides a robust architecture for developing MCP servers, offering advanced features such as dependency injection, modular organization, and type-safe tooling out of the box.

Designed for enterprise applications, NitroStack enforces architectural best practices, ensuring your codebase remains clean, testable, and adaptable as it scales from a single tool to a complex distributed system.

## Core Architecture

### Decorator-Based Design

NitroStack utilizes a descriptive, decorator-based architecture. This approach reduces boilerplate and improves readability by keeping configuration close to the implementation.

### Key Advantages

- **Declarative Syntax:** Define behavior and configuration using clear, semantic decorators.
- **Type, Schema & Validation:** Native TypeScript integration with Zod schemas ensures end-to-end type safety and runtime validation.
- **Composability:** Apply multiple behaviors—such as authentication, caching, and rate limiting—via composable decorators.
- **Developer Experience:** leveraging TypeScript's metadata reflection for robust IntelliSense and refactoring support.

## Example Implementation

```

@Tool({
  name: 'search_products',
  description: 'Search product catalog with advanced filtering specific',
  inputSchema: z.object({
    query: z.string(),
    category: z.string().optional(),
    filters: z.object({
      minPrice: z.number().optional(),
      maxPrice: z.number().optional(),
    }).optional(),
    pagination: z.object({
      page: z.number().default(1),
      limit: z.number().default(20)
    }).default({})
  })
})
@UseGuards(JwtAuthGuard)
@UseMiddleware(AuditLoggingMiddleware)
@Cache({ ttl: 600, key: (input) => `search:${input.query}` })
@RateLimit({ requests: 100, window: '1h' })
@Widget('product-grid-view')
async searchProducts(input: any, ctx: ExecutionContext) {
  ctx.logger.info('Executing product search', { query: input.query });
}
```

```
    return await this.productService.executeSearch(input);  
  }  
}
```

This single declaration encapsulates API definition, validation, security, observability, performance optimization, and UI presentation layer association.

## Modular Organization

NitroStack enforces a modular architecture to promote separation of concerns and maintainability. Applications are comprised of self-contained modules that encapsulate related capabilities, providers, and context.

### Module Definition

● ● ● TYPESCRIPT



```
@Module({  
  name: "commerce-engine",  
  description: "Core commerce and product management domain",  
  controllers: [ProductTools, InventoryResources, PricingPrompts],  
  providers: [ProductService, InventoryService, PricingEngine],  
  imports: [DatabaseModule, AuthenticationModule],  
  exports: [ProductService]  
})  
  
export class CommerceModule {}
```

### Architectural Benefits

- **Encapsulation:** Strict boundaries between domains (e.g., User Management, Payment Processing).
- **Reusability:** Modules can be exported and shared across different services.
- **Dependency Management:** Explicit definition of module dependencies and public interfaces.
- **Testability:** Modules can be isolated for unit and integration testing.

## Dependency Injection (DI)

At the core of NitroStack is a sophisticated Dependency Injection container. This system manages the lifecycle of application components, resolving dependencies automatically and promoting loose coupling.

### DI Implementation

```
TYPESCRIPT

@Injectable()
export class OrderService {
  constructor(
    private readonly repository: OrderRepository,
    private readonly paymentGateway: PaymentGateway,
    private readonly notificationService: NotificationService
  ) {}

  async processOrder(order: OrderDto): Promise<OrderResult> {
    const transaction = await this.paymentGateway.authorize(order.paymentInfo);
    const result = await this.repository.save({ ...order, transactionId: transactionId });
    await this.notificationService.notifyConfirmation(result);
    return result;
  }
}
```

```
}  
}
```

The DI system supports:

- **Singleton, Transient, and Scoped lifecycles**
- **Factory Providers** for dynamic instantiation
- **Value Providers** for configuration injection
- **Circular Dependency Resolution**

## Security & Guards

Security is implemented via a declarative Guard system. Guards intercept execution contexts to validate requests, manage authentication, and enforce authorization policies before business logic is invoked.

● ● ● TYPESCRIPT



```
@Tool({ name: 'admin_dashboard' })  
@UseGuards(OAuth2Guard, RoleGuard.for(['admin', 'super-admin']))  
async accessDashboard(input: any, ctx: ExecutionContext) {  
  // Logic executes only if all guards pass  
  return this.dashboardService.getMetrics();  
}
```

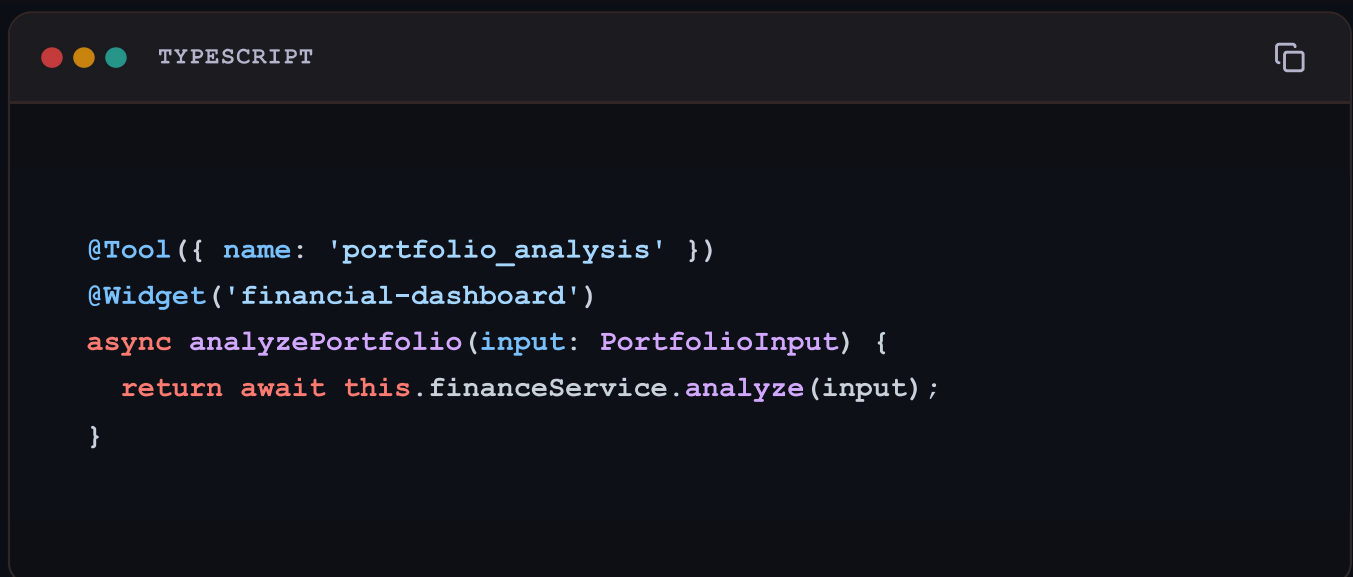
Included security primitives:

- **Authentication:** JWT, OAuth 2.1, API Key strategies.

- **Authorization:** Role-based (RBAC) and Attribute-based (ABAC) access control.
- **Scope Management:** Fine-grained permission scopes for secure tool execution.

## Intelligent UI Widgets

NitroStack bridges the gap between backend logic and frontend presentation with its Widget system. Tools can define associated UI components using React and Next.js, which are rendered by compatible clients (such as NitroStack Studio).



```
@Tool({ name: 'portfolio_analysis' })
@Widget('financial-dashboard')
async analyzePortfolio(input: PortfolioInput) {
  return await this.financeService.analyze(input) ;
}
```

Widgets leverage standard web technologies (React, Tailwind CSS) to provide rich, interactive interfaces for AI model outputs, enhancing the user experience beyond simple text responses.

## Ecosystem Features

NitroStack provides a comprehensive suite of enterprise-ready capabilities:

- **Middleware Pipeline:** Intercept and transform requests and responses globally or per-route.
- **Interceptors:** Advanced execution flow control and response mapping.
- **Pipes:** Reusable input validation and transformation logic.
- **Exception Filters:** Centralized error handling and standardized response formatting.
- **Observability:** Integrated structured logging and event systems.
- **NitroStack Studio:** A dedicated environment for developing, testing, and debugging MCP servers and widgets.

## Why NitroStack?

### For Enterprise Development

- **Standardization:** Enforces consistent patterns across large teams.
- **Reliability:** Built on proven architectural patterns for long-term maintainability.
- **Security:** Security-first design with robust authentication and authorization mechanisms.

### For AI Integration

- **Context Protocol Native:** Built specifically for the Model Context Protocol specification.
- **Rich Interaction:** Delivers structured data and interactive UI components to AI agents.
- **Scalable Context:** Efficiently manages tools and resources for complex agentic workflows.

---

## Next Steps

- [Installation](#) – Setup your development environment.
- [Quick Start](#) – Initialize your first NitroStack project.
- [CLI Introduction](#) – Learn about the CLI tools.
- [Server Architecture](#) – Deep dive into server concepts.